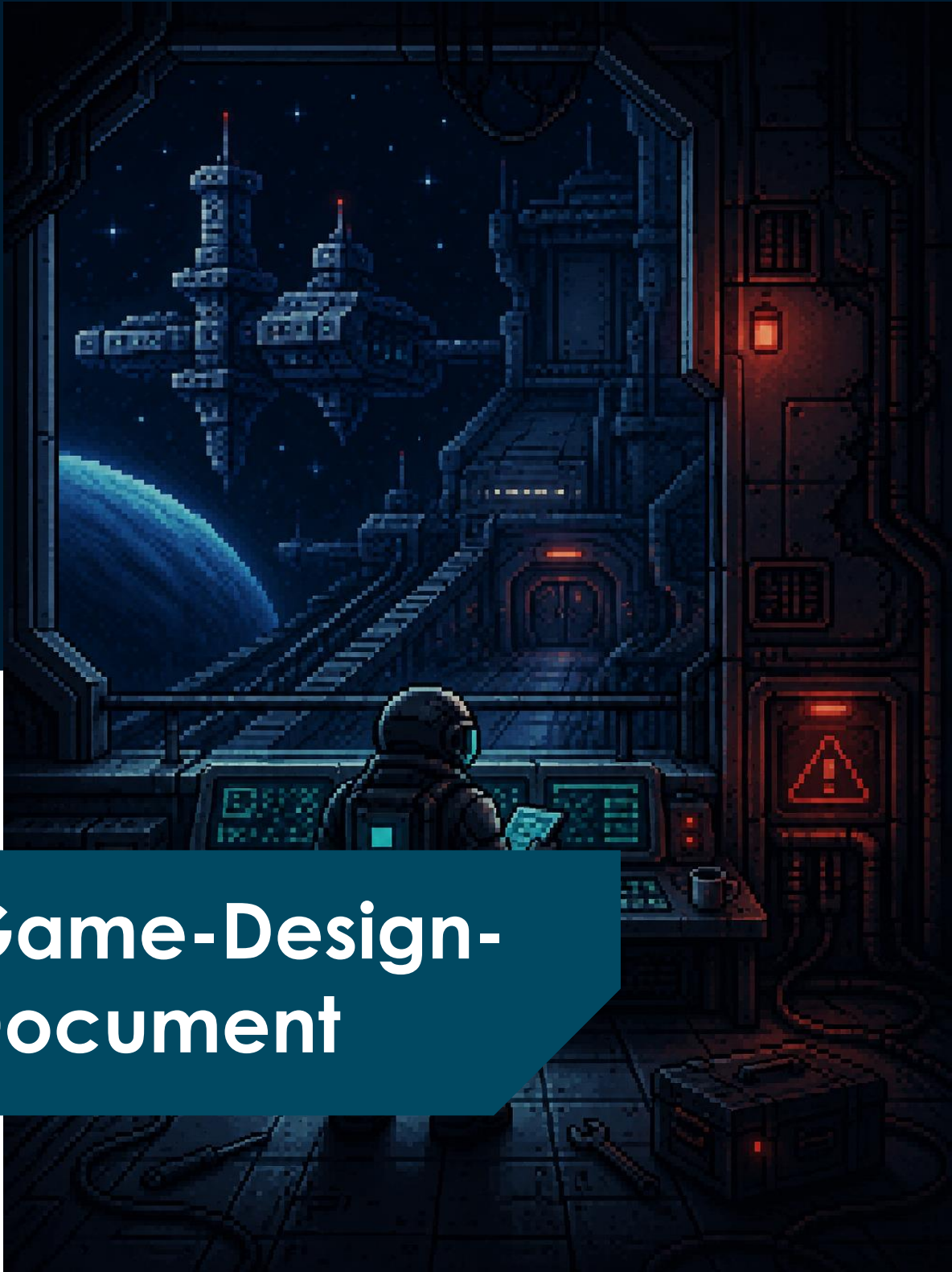




Game-Design-Document

Projektname: Deep Space Debugger

Ersteller: Franz Hielscher

INHALTSVERZEICHNIS

1.	PROJEKTÜBERSICHT	4
1.1	Projektbeschreibung	4
1.2	Mission Statement	4
1.3	Unique Selling Point	5
1.4	Zielgruppenanalyse	5
1.5	Ziele	7
1.6	Vision	8
2.	PROJEKTPLANUNG	9
2.1	Erläuterung zur Planung	9
2.2	Gantt Diagramm	10
3.	RISIKEN	11
3.1	Technische Herausforderungen	11
3.2	Zielgruppenakzeptanz	11
3.3	Zeitliche Risiken	12
4.	TECHNISCHE SPEZIFIKATION	13
4.1	Zielplattformen	13
4.2	Engine / Tools	13
4.3	Technologie: Objektorientierte Spielarchitektur	14
5.	AUDIO UND MUSIK	15
5.1	Soundeffekte	15
5.2	Musikstil	15
5.3	3D-Audio	15
6.	GRAFIK UND ASSETS	17
6.1	Visueller Stil	17
7.2	Umgebungsdesign	17
7.3	Charakterdesign	17
7.4	Interaktive Elemente	18
7.	Interaktionsdesign	19
7.1	Controller-Input	19
7.2	Physische Mechaniken	20
8.	Spielerfahrung	22
8.1	Immersionselemente	22
8.2	Bewegungssystem	22

8.3	UI/UX Design	23
8.4	Komfortmaßnahmen gegen Motion-Sickness	23
9.	Spielverlauf	24
9.1	Grundstruktur: Lineare Missionskette	24
9.2	Core-Game-Loop	24
9.3	Fortbewegung und Übergänge	25
9.4	Abschluss	25
10.	Story & Setting	26
10.1	Kern-Setting	26
10.2	Wichtige Orte (Levels)	26
10.3	Story & Motivation	26
11.	MOODBOARDS UND IMPRESSIONEN	28
11.1	Räume der HTWK Leipzig	28
11.2	Farb- und Lichtabstimmung	29
11.3	UI / UX und interaktive Elemente	30
13	QUELLEN	31
13.1	Bildquellen	31

1. PROJEKTÜBERSICHT

1.1 Projektbeschreibung

Das Spiel **Deep Space Debugger** ist ein prototypisches **2D-Serious-Game** zur spielerischen Vermittlung grundlegender Konzepte des objektorientierten Paradigmas (OOP). Das Herzstück des Projekts ist eine interaktive Lernumgebung im **Science-Fiction-Setting einer beschädigten Raumstation**, in der Spielende verschiedene Systeme analysieren, reparieren und schrittweise wiederherstellen müssen.

Die Anwendung setzt auf eine klare **Verbindung von Gameplay, Rätseldesign und didaktischen Lernmechaniken**, um theoretische Inhalte des OOP verständlich und motivierend zu vermitteln. Das visuelle Fundament basiert auf einer **stilisierten 2D-Pixeloptik** und wird ergänzt durch:

- **Interaktive Debugging- und Rätselmechaniken.**
- **Gameplaybasierte Vermittlung von OOP-Konzepten** wie Klassen, Objekten, Vererbung und Polymorphie.
- **Spielerische Exploration und Freischaltung neuer Stationsbereiche.**
- **Atmosphärisches Audio- und UI-Design** zur Unterstützung der Immersion.

Der Fokus liegt auf der spielerischen Unterstützung von Lernprozessen durch interaktives Problemlösen und Anwendung objektorientierter Prinzipien. Ziel des Projekts ist die prototypische Entwicklung eines motivierenden Lernansatzes, der theoretische Programmierkonzepte durch direkte spielerische Erfahrung greifbar macht.

1.2 Mission Statement

Deep Space Debugger verfolgt das Ziel, grundlegende Konzepte der objektorientierten Programmierung durch **spielerische Interaktion** verständlicher und motivierender zu vermitteln. Durch exploratives Gameplay, interaktive Rätselmechaniken und die praktische Anwendung von OOP-Prinzipien soll Studierenden der Zugang zu abstrakten Konzepten wie Klassen, Vererbung und Polymorphie erleichtert werden. Das Spiel versteht sich dabei als ergänzender Lernansatz zu klassischen Lehrmethoden und verbindet theoretische Inhalte mit aktiver Problemlösung in einer interaktiven Spielumgebung.

1.3 Unique Selling Point

Der besondere Ansatz von **Deep Space Debugger** liegt in der direkten Verknüpfung von Gameplay und der Anwendung des objektorientierten Paradigmas. OOP-Konzepte werden nicht nur erklärt, sondern aktiv als Spielmechaniken erfahrbar gemacht und in eine interaktive Science-Fiction-Lernumgebung eingebettet.

1.4 Zielgruppenanalyse

Ausgangslage

Die Zielgruppe ergibt sich aus der Problemstellung des Projekts. Studierende technischer und informatischer Studiengänge haben häufig Schwierigkeiten, abstrakte Konzepte der OOP nachhaltig zu verstehen und praktisch anzuwenden.

Primäre Zielgruppe: Studierende technischer und informatischer Studiengänge

Die primäre Zielgruppe umfasst Studierende, die sich im Rahmen ihres Studiums mit den Konzepten des OOP beschäftigen. Dazu zählen insbesondere:

- Informatikstudierende
- Medieninformatikstudierende
- Studierende technischer Studiengänge
- Teilnehmende an Programmierkursen

Sekundäre Zielgruppe: Selbstlernende und programmierinteressierte Personen

Zusätzlich richtet sich das Projekt an:

- Hobbyentwickler:innen
- Selbstlernende
- Personen mit Interesse am Game-Based-Learning
- Programmieranfänger:innen außerhalb akademischer Kontexte

Merkmal	Beschreibung	Relevanz für das Design
Alter	ca. 18 bis 36 Jahre	Zielgruppe ist mit digitalen Medien und Computerspielen vertraut; modernes und intuitives UI-Design erforderlich

Bildungskontext	Studierende, Selbstlernende und programmierinteressierte Personen	Lerninhalte müssen sowohl akademisch relevant als auch allgemein verständlich vermittelt werden
Vorwissen	Grundkenntnisse in Programmierung, Unsicherheit bei OOP-Konzepten	Komplexe Konzepte müssen visuell, interaktiv und schrittweise vermittelt werden
Tech-Affinität	Mittel bis hoch; Erfahrung mit Games, Software und digitalen Lernmedien	Ermöglicht gameplaybasierte Lernmechaniken und experimentelles Problemlösen
Spielerfahrung	Gelegenheitsspieler bis erfahrene Spieler	Steuerung und Gameplay sollten leicht verständlich bleiben, aber dennoch motivierende Interaktionen bieten
Lernstil	Visuell, interaktiv und explorativ	Rätsel, Exploration und direkte Systeminteraktion fördern „Learning by Doing“
Motivation	Lernen und Unterhaltung kombiniert	Serious-Game-Elemente sollen Motivation und Lernbereitschaft hinsichtlich OOP steigern
Typische Lernprobleme	Schwierigkeiten beim Verständnis abstrakter OOP-Konzepte	Gameplay muss theoretische Inhalte praktisch erfahrbar machen
Nutzungskontext	Studium, Selbstlernen oder ergänzende Lernunterstützung	Flexible und kompakte Spielerfahrung notwendig
Aufmerksamkeits-spanne	Begrenzte verfügbare Lernzeit	Kurze, kompakte Spielerfahrung (ca. 15 bis 20 Minuten) mit klarer Progression
Plattformnutzung	PC / Laptop, Mobile	Kompatibilität für eine Vielzahl an Geräten

1.5 Ziele

Die Ziele leiten sich direkt aus der Mission ab, abstrakte Konzepte der objektorientierten Programmierung durch spielerische Interaktion verständlicher und motivierender zu vermitteln.

Didaktische Ziele (Game-Based-Learning & OOP)

- **Verständnis abstrakter OOP-Konzepte:** Spieler:innen sollen grundlegende Prinzipien des objektorientierten Paradigmas wie Klassen, Objekte, Vererbung und Polymorphie durch interaktive Spielmechaniken besser verstehen und anwenden können.
- **Praxisnaher Wissenstransfer:** Theoretische Inhalte sollen nicht nur textbasiert vermittelt, sondern durch gameplayorientierte Problemlösung direkt erfahrbar gemacht werden.
- **Learning by Doing:** Durch Exploration, Rätselmekaniken und Systeminteraktionen soll aktives und experimentelles Lernen gefördert werden.
- **Motivation und Lernmanagement:** Die Kombination aus Science-Fiction-Setting, Pixelart-Ästhetik und spielerischen Herausforderungen soll die Motivation steigern und den Zugang zu komplexen Lerninhalten erleichtern.
- **Niedrige Einstiegshürde:** Das Spiel soll OOP-Konzepte verständlich vermitteln, ohne umfangreiche Programmierkenntnisse oder klassische Codeeingabe vorauszusetzen.

Spielerische Ziele (Gameplay & Experience)

- **Interaktive Problemlösung:** Spieler:innen analysieren fehlerhafte Systeme, erkennen Zusammenhänge und reparieren logische Fehler innerhalb der Raumstation.
- **Exploration und Progression:** Neue Bereiche der Raumstation werden schrittweise durch gelöste Herausforderungen freigeschaltet.
- **Verknüpfung von Gameplay und Lerninhalten:** OOP-Konzepte sollen direkt in die Kernmechaniken des Spiels integriert werden.

Technologische Ziele (Entwicklung & Umsetzung)

- **Prototypische Umsetzung eines 2D-Serious-Games:** Entwicklung eines spielbaren Minimum Viable Products (MVP) mit zentralen Gameplay- und Lernmechaniken.
- **Modulare Systemarchitektur:** Aufbau eines objektorientierten Spielsystems, dass die vermittelten OOP-Konzepte auch technisch innerhalb der Softwarearchitektur widerspiegeln.

- **Technische Umsetzung in der Godot Engine:** Einsatz der Engine zur Entwicklung eines plattformunabhängigen 2D-Spiels mit Fokus auf Performance, Modularität und Erweiterbarkeit
- **Anwendung des MDA-Frameworks:** Strukturierte Konzeption der Mechanics, Dynamics und Aesthetics zur Verbindung von Gameplay, Lernerfahrung und Nutzerinteraktion.

1.6 Vision

Deep Space Debugger verfolgt die Vision, abstrakte Konzepte der objektorientierten Programmierung durch **interaktive und spielerische Lernansätze** zugänglicher zu machen. Das Projekt versteht sich als **prototypischer Beitrag** zur **Verbindung von Game-Based-Learning und Informatiklehre** und untersucht, wie spielerische Interaktion das Verständnis komplexer Programmierkonzepte unterstützen kann.

Langfristig soll gezeigt werden, wie Serious Games klassische Lehrmethoden sinnvoll ergänzen und Lernprozesse durch Motivation, Exploration und aktive Problemlösung fördern können.

2. PROJEKTPLANUNG

2.1 Erläuterung zur Planung

Die Projektplanung erfolgt strukturiert anhand definierter Arbeitspakete und Meilensteine. Dabei werden sowohl die wissenschaftliche Ausarbeitung der Bachelorarbeit als auch die prototypische Entwicklung des Serious Games **Deep Space Debugger** berücksichtigt.

Der Fokus liegt auf der schrittweisen Konzeption, technischen Umsetzung und Evaluation eines spielbaren Prototyps, der zentrale Konzepte der objektorientierten Programmierung durch gameplaybasierte Mechaniken vermittelt.

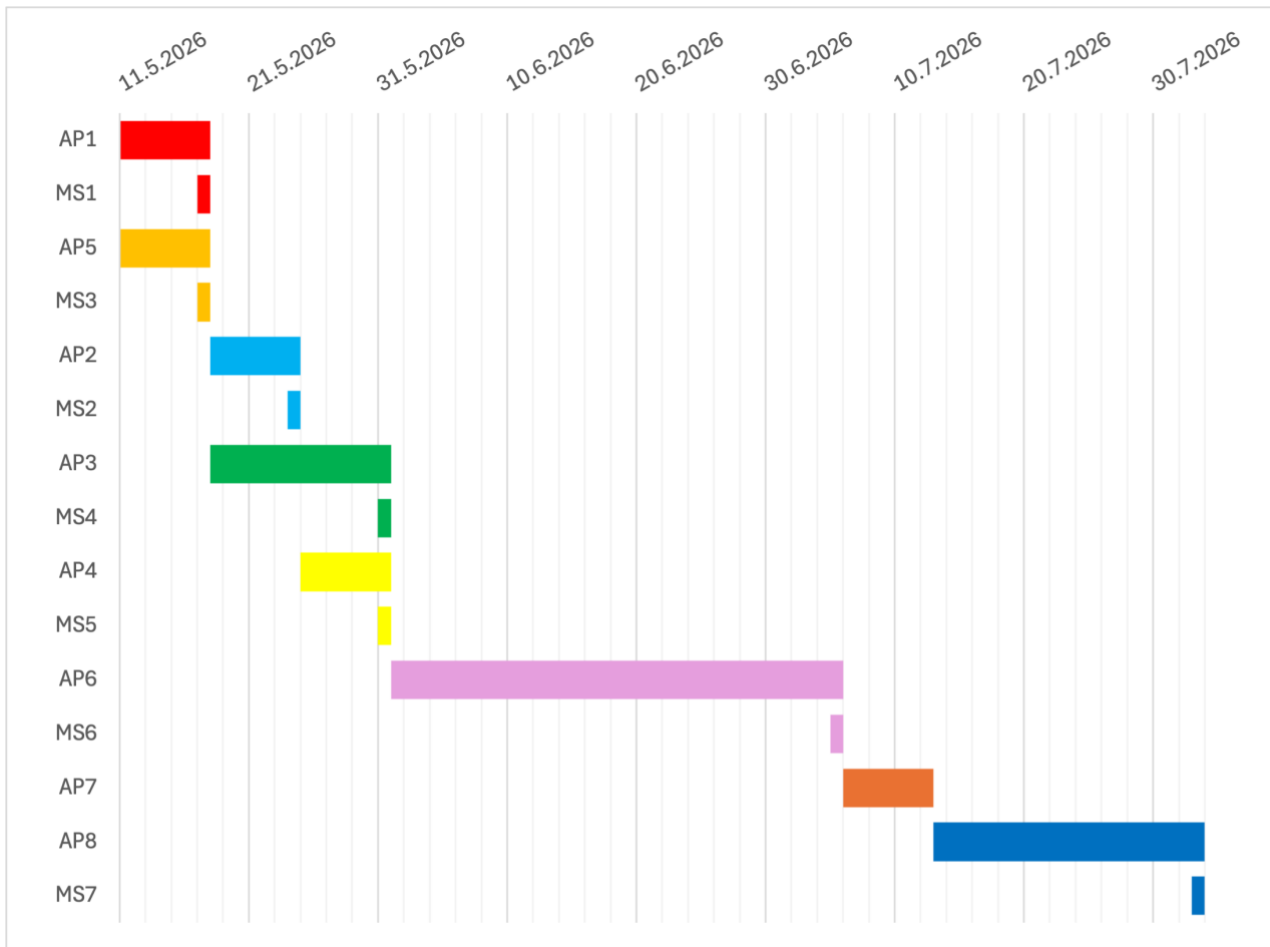
Meilensteine

- MS1 - Thema & Forschungs-/Spielkonzept definiert
- MS2 - Experteninterview durchgeführt & ausgewertet
- MS3 - Projektsetup (GitHub & Godot Engine) abgeschlossen
- MS4 - Literaturbasis erstellt & strukturiert
- MS5 - Game-Design-Dokument fertiggestellt
- MS6 - Spiel als spielbarer Prototyp (MVP) fertiggestellt
- MS7 - Abgabeverision der Bachelorarbeit fertiggestellt

Arbeitspakete

- AP1 - Planung & Organisation
- AP2 - Experteninterview
- AP3 - Literaturrecherche & Theorie
- AP4 - Game Design & Konzeption
- AP5 - Projektsetup (GitHub & Godot Engine)
- AP6 - Prototyp-Entwicklung (Mechaniken, grundlegende Spiellogik & Audio)
- AP7 - Testing & Optimierung
- AP8 - Finalisierung Bachelorarbeit

2.2 Gantt Diagramm



3. RISIKEN

Bei der prototypischen Entwicklung des Serious Games Deep Space Debugger ergeben sich verschiedene Risiken, die sowohl die **technische Umsetzung** als auch die **didaktische Gestaltung** sowie den **zeitlichen Projektverlauf** betreffen. Damit der Projekterfolg sichergestellt werden kann, ist es wichtig, dass **potenzielle Herausforderungen frühzeitig erkannt** und bei der **Planung berücksichtigt** werden. Die wesentlichen Risikobereiche umfassen technische Herausforderungen, Zielgruppenakzeptanz und zeitliche Risiken.

3.1 Technische Herausforderungen

Gameplay- und Systementwicklung:

- Fehleranfälligkeit bei komplexen Interaktionen zwischen verschiedenen Spielsystemen
- Bugs innerhalb der Rätsel-, Trigger- und Debugging-Mechaniken
- Fehlende Quellassets
- Schwierigkeiten bei der Umsetzung modularer OOP-basierter Spielsysteme

Technische Umsetzung in der Godot Engine:

- Probleme bei der Integration verschiedener Spielmechaniken
- Unerwartete Engine- oder Scriptfehler
- Risiko von Performanceproblemen bei zu vielen interaktiven Objekten

UI- und Debugging-System:

- Erhöhter Entwicklungsaufwand für ein verständliches UI- und Debugging-Interface
- Risiko einer zu umfangreichen und unübersichtlichen UI
- Komplizierte Balance zwischen Spielbarkeit und didaktischer Funktion

3.2 Zielgruppenakzeptanz

Vermittlung der Lerninhalte:

- Gefahr, dass die OOP-Konzepte weiterhin als zu abstrakt wahrgenommen werden
- Risiko, dass die Lerninhalte zu stark vereinfacht werden
- Unterschiedliche Vorkenntnisse innerhalb der Zielgruppen
- Kernmechaniken können die vorhandenen OOP-Konzepte nicht klar genug vermitteln

Motivation & Gameplay:

- Gefahr von Abbrüchen oder Nichtnutzung aufgrund fehlender Motivation
- Risiko, dass Spielmechaniken als zu repetitiv wahrgenommen werden
- Überforderung durch zu komplexe kombinierte Rätsel- und Lernmechaniken

Zugänglichkeit & Verfügbarkeit:

- Unterschiedliche Spielerfahrungen und technische Kenntnisse
- Risiko, dass die Steuerung oder Benutzeroberfläche zu komplex sind
- Begrenzte Zeitbereitschaft für Lernspiele innerhalb des Studienalltags

3.3 Zeitliche Risiken**Konzeption & Planung:**

- Hoher Aufwand bei Literaturrecherche und theoretischer Ausarbeitung
- Verzögerungen durch Anpassungen des Game-Designs
- Möglicher negativer Einfluss / zusätzliche Abstimmungen durch das geplante Experteninterview

Projektumfang:

- Risiko eines zu großen Scopes für die verfügbare Bearbeitungszeit
- Gefahr, dass geplante Features gekürzt / reduziert werden müssen
- Begrenzte Entwicklungszeit im Rahmen der Bachelorarbeit

Entwicklung, Optimierung & Testing:

- Zeitintensive Implementierung von Gameplaymechaniken und UI-Systemen
- Unerwarteter Aufwand bei der Fehlersuche und der Optimierung
- Umfangreiche Testphasen notwendig, um Fehler zu vermeiden

Externe Abhängigkeiten:

- Technische Probleme mit Entwicklungssoftware oder Hardware
- Zeitliche Überschneidung mit anderen Verpflichtungen
- Mögliche Verzögerungen durch Versions- oder Pluginprobleme innerhalb der Engine
- Mögliche Verzögerungen durch das Prüfungsamt

4. TECHNISCHE SPEZIFIKATION

4.1 Zielplattformen

Die Serious Game Deep Space Debugger wird als **plattformübergreifende 2D-Anwendung** mithilfe der **Godot Engine** entwickelt. Ziel ist eine möglichst flexible Nutzbarkeit auf verschiedenen Endgeräten, um den Zugang für unterschiedliche Zielgruppen zu gewährleisten.

Primäre Zielplattformen:

- Desktop-PCs und Laptops (Windows / MacOS / Linux)
- Mobile Endgeräte (Android / iOS)
- Steuerung über Tastatur und Maus sowie Touch-Eingaben auf mobilen Geräten

Kompatibilität & Erweiterbarkeit:

- Plattformunabhängige Entwicklung innerhalb der **Godot Engine**
- Fokus auf **modulare** und **skalierbare Projektstruktur**
- Möglichkeit zukünftiger Erweiterungen und zusätzlicher Plattformanpassungen im Rahmen weiterer Entwicklungsphasen

4.2 Engine / Tools

Game-Engine:

- **Godot Engine** (Nutzung der aktuellen Version, Stand: 11.05.2026) als zentrale Entwicklungsumgebung
- Programmierung primär mit **GScript**
- Entwicklung als **2D-Projekt** mit plattformübergreifender Unterstützung

Gameplay- & Systementwicklung:

- Nutzung des integrierten Node- und Szenensystems der Engine
- Modularer Aufbau der Spielsystem orientiert an OOP
- Entwicklung interaktiver Rätsel-, Trigger- und Debugging-Mechaniken

2D- und Asset-Erstellung / -Bearbeitung:

- Erstellung und Bearbeitung von Pixelart-Grafiken
- Nutzung einfacher **2D-Asset-Pipelines** für Charaktere, Umgebungen und UI-Elemente
- Fokus auf stilisierte **Science-Fiction-Pixeloptik**

Audio-Tools:

- **Reaper / Audacity** zur Bearbeitung, Normalisierung und zum Export der Soundeffekte und Hintergrundmusik
- **Nutzung lizenzfreier Soundbibliotheken** (bspw. Freesound) als kostenfreie Quellen für Soundeffekte

Versionskontrolle:

- **GitHub** als primäres System für Versionsverwaltung und Projektorganisation

4.3 Technologie: Objektorientierte Spielarchitektur

Definition und Funktionsweise

Die technische Struktur von Deep Space Debugger basiert auf einer **modularen objektorientierten Spielarchitektur**. Zentrale Spielsysteme und Interaktionsobjekte werden dabei als eigenständige, wiederverwendbare Komponenten umgesetzt.

Das System orientiert sich an grundlegenden Prinzipien der objektorientierten Programmierung:

- **Klassen und Objekte**
- **Vererbung**
- **Polymorphie**
- **Objektbeziehungen und Komposition**

Interaktive Elemente wie Türen, Terminals oder Sicherheitssysteme besitzen **eigene Zustände und Verhaltensweisen** und reagieren **dynamisch** auf Spielerinteraktionen.

Nutzen für das Projekt

- Direkter Praxisbezug → Struktur spiegelt vermittelte OOP-Konzepte wider.
- Modulare Entwicklung & Wiederverwendbarkeit → flexible Erweiterung / Anpassung.
- Verständlichkeit → OOP-Konzepte werden spielerisch erfahrbar gemacht.

5. AUDIO UND MUSIK

5.1 Soundeffekte

Soundeffekte sollen das Nutzererlebnis unterstützen und Orientierung geben. Sie bleiben dezent, damit sie nicht ablenken, aber realitätsnah und deutlich genug für Feedback.

Typen von Soundeffekten:

- Interaktionssounds:
 - Leises, sanftes Klicken bei Menüs
- Umgebungsgeräusche:
 - Mensa: leises / lautes Stimmenmurmel, Schritte
 - Mocap Labor: leises Summen der Computer, Geräusch Ball trifft Korb
 - Bibliothek: Tastaturtippen, Seiten umblättern, Buch aufschlagen
 - VR-Labor: Tastaturtippen, leises Summen der Computer, gedämpfte Stimmen vom Flur aus
- Objektinteraktionen:
 - Kurze, weiche Sounds beim Aufnehmen / Ablegen / Aktivieren von Objekten

5.2 Musikstil

Die Musik soll je nach Umgebung als angenehmer Hintergrund dienen und nicht im Mittelpunkt stehen. Sie schafft Atmosphäre und begleitet Nutzer durch den Campus.

Stil:

- Atmosphärische, ruhige Melodien, unaufdringliche Ambient-Musik
- Moderne, leicht elektronische Klänge
- Keine auffälligen Beats oder drastische Lautstärkewechsel
- Sanfte Überblendungen zwischen verschiedenen Umgebungen

5.3 3D-Audio

Der Einsatz von 3D-Audio soll der räumlichen Orientierung, besseren Wahrnehmung von Entfernungen sowie einer realistischen Immersion dienen.

Einsatz:

- Bewegungsabhängige Geräusche:
 - Positionsänderung: Geräusche in der Umgebung verändern sich je nachdem, wo der Nutzer steht und wohin er sich bewegt (bspw. Murmeln wird leiser / lauter)
 - Schritte anderer Personen
 - Objektbewegung: wenn der Nutzer ein Objekt bewegt, so bewegt sich der jeweilige Sound ebenfalls
- Räumliche Effekte:
 - Positionsbezogene Soundquellen: Soundquellen sind pro Raum verankert und aus einer bestimmten Richtung hörbar
 - Entfernungswahrnehmung: lauter, wenn näher und leiser, wenn weiter entfernt
 - Orientierungshilfen
 - Gedämpfte Sounds bei Geräuschen außerhalb des Raumes
 - Hall- und Raumklangsimulation: einzelne Räume haben ein unterschiedliches akustisches Verhalten (bspw. Mensa mehr Hall als VR-Labor)

6. GRAFIK UND ASSETS

6.1 Visueller Stil

Der visuelle Stil der VR-Anwendung soll modern, freundlich und gut lesbar sein. Wichtig ist hierbei eine klare, realitätsnahe Darstellung der HTWK-Umgebung, ohne zu viele Details, damit die Performance auf den VR-Headsets stabil bleibt.

Zentrale Stilmerkmale:

- Realitätsnah, aber leicht vereinfacht (Low-Poly / Mid-Poly) zur Performanceoptimierung
- Klare Formen, gut erkennbare Räume, helle und einladende Farben (positives Erlebnis für den Nutzer)
- Einheitliche Material- und Farbgestaltung, orientiert an der HTWK
- Fokus auf Lesbarkeit und Raumorientierung statt fotorealistischer Texturen
- Stil soll den realen Campus und dessen wichtigsten Räume wiedererkennbar darstellen, aber gleichzeitig angenehm ruhig und übersichtlich bleiben für den Nutzer

7.2 Umgebungsdesign

Das Umgebungsdesign basiert auf den Bereichen der HTWK Leipzig (z.B. Mensa, Bibliothek, VR-Labor, usw.).

Schwerpunkte:

- Originalgetreue Layouts und Raumstrukturen
- Wichtige Orte und Orientierungspunkte klar hervorgehoben
- Verwendung von leichtgewichtigen Assets zur Performanceoptimierung
- Beleuchtung neutral und realistisch
- Räume enthalten die notwendigsten Objekte

7.3 Charakterdesign

Da es bei der VR-Anwendung primär um Räume und Orientierung geht, werden Charaktere nicht in Betracht gezogen.

7.4 Interaktive Elemente

Interaktive Elemente sollen intuitiv, klar markiert und leicht bedienbar sein.

Beispiele:

- Hervorgehobene Interaktionsobjekte: Türen, Infopunkte, Tafeln, usw.
- Symbole: vereinheitlichte Markierungen für z.B. „Klicken“, „Aufnehmen“, „Ablegen“, usw.
- Objekte zum Anfassen: Bücher, Geräte, einfache Objekte, Studentenausweis, Geldschein
- Attachpoints Objekte am Automaten werden bei Annäherung des Objektes farblich hervorgehoben
- UI-Elemente: minimalistische VR-UI im Raum

7. INTERAKTIONSDESIGN

Das Interaktionsdesign orientiert sich am Ziel, eine **niederschwellige, intuitive VR-Erfahrung** für Nutzer zu schaffen. Die Spieler*innen erleben alle Räume aus der **First-Person-Perspektive** in fotorealistischen 3D-GS-Szenen und bewegen sich mithilfe der VR-Controller durch die Umgebung.

7.1 Controller-Input

Die **VR-Controller** der Meta Quest 3S sind das einzige und zentrale Eingabemedium. Alle Interaktionen sind vollständig controllerbasiert, um eine robuste Bedienung zu gewährleisten.

Grundinteraktionen:

- Fortbewegung im Raum (Smooth Locomotion):
 - Standardmäßig steht den Spieler*innen **Smooth Locomotion** über den **linken Thumbstick** zur Verfügung:
 - nach vorne/hinten: Vorwärts- bzw. Rückwärtsbewegung
 - nach links/rechts: Seitwärtsbewegung

Rotation (Smooth Turn):

- Zusätzlich zur natürlichen Kopfbewegung kann die Kamera über den **rechten Thumbstick** gedreht werden
- Die Rotation erfolgt als **kontinuierliche, analoge Rotation (Smooth Turn)** - es werden **keine Snap-Turns** verwendet

Objektinteraktion:

- Trigger-Taste (Zeigefinger):
 - Auslösen von Interaktionspunkten (z. B. Infotafeln, Schaltflächen, Menüs)
 - Bestätigen von Auswahloptionen innerhalb der VR-UI
- Grip-Taste:
 - Aufnehmen und Ablegen einfacher interaktiver Objekte (z. B. Studentenausweis und Geldschein, welche in den Automaten eingeführt werden)
- A/X-Buttons:
 - Kontextsensitive Aktionen (z. B. „Quest-Hinweis anzeigen“, „Interaktion starten“)

- B/Y-Buttons:
- Öffnen eines einfachen In-Game-Menüs (z. B. Einstellungen, Komfortoptionen, Hilfetexte)
- UI-Interaktion (In-World-UI):
 - Interaktion mit Benutzeroberflächen erfolgt über **Controller-basierte Ray-Interaktion**
 - Anklickbare Elemente werden durch Hover-Zustände und subtile Animationen hervorgehoben, um die Lesbarkeit und Benutzbarkeit zu erhöhen
- Feedback:
 - **Haptisches Feedback** (Vibration) bei erfolgreichen Interaktionen, z. B.:
 - Knopfdruck
 - Einsammeln eines Objekts
 - Abschluss eines Quest-Schrittes
 - Ergänzend kurze, dezente **Soundeffekte** zur akustischen Bestätigung des Spielerhandelns

7.2 Physische Mechaniken

Umgebungsphysik und Kollisionsverhalten:

- Die **Splatting-Modelle** dienen ausschließlich der visuellen Darstellung und besitzen selbst **keine Kollisionen**
- Parallel dazu wird eine **vereinfachte Collider-Geometrie** (Box- und ggf. Mesh-Collider) aufgebaut, die:
 - Wände, Böden, Türen und relevante Möbelstücke abbildet,
 - verhindert, dass Spieler durch sichtbare Strukturen „hindurchlaufen“
 - als Basis für Trigger-Events (z. B. Betreten eines Quest-Bereichs) dient
- Kopf-/Körperkollisionen werden vorsichtig eingesetzt: Die Bewegung des Spielers wird sanft gestoppt, ohne harte Block-Effekte, um Motion Sickness zu vermeiden

Interaktive Objekte und Trigger:

- Wichtige Interaktionspunkte in den Rekonstruktionen (z. B. Mensa-Kasse, PC-Arbeitsplätze, Laboreinrichtungen) erhalten:

- eine **visuelle Markierung** im 3D-GS-Raum (Icon, leichtes Aufleuchten, Markierung)
- einen **unsichtbaren Collider/Trigger** im Unity-Raum für die Logik
- Beim Betreten eines Triggerbereichs oder gezielten Auslösen per Controller werden:
 - Informations-Overlays, oder Quest-Fortschritt angezeigt

Physische Quest-Mechaniken:

- Sammeln, Aktivieren oder Auslösen von Objekten unterstützt das didaktische Ziel der Anwendung, z. B.:
 - „Aktiviere ein bestimmtes Gerät“
 - „Starte eine Informationssequenz an einem markierten Punkt“
- Die Mechaniken bleiben bewusst **einfach und klar lesbar**, damit der Fokus auf Orientierung, Verständnis der Infrastruktur und Informationsvermittlung liegt

8. SPIELERFAHRUNG

Die Spielerfahrung steht im Zentrum des Projektes, da sie maßgeblich darüber entscheidet, wie gut die Nutzer die virtuellen Räume aufnehmen und letztendlich von der VR-Anwendung profitieren. Die Gestaltung dieser Erfahrung setzt sich aus mehreren Bereichen zusammen. Jeder dieser Bereiche wird so ausgelegt, dass sie dem Anwender spielerisch motivieren und ein verträgliches VR-Erlebnis gewährleisten.

8.1 Immersionselemente

Damit der Anwender vollständig in das Erlebnis eintauchen kann verwenden wir verschiedene Methoden der Immersion. Ein großer Anteil dabei hat das angestrebte Raumgefühl, was durch folgende Aspekte definiert ist:

- Realistische Innenraumdarstellung
 - umgesetzt durch 3D Gaussian Splatting: Diese Technologie verstärkt durch seine besondere Detailtreue das Gefühl, wirklich körperlich anwesend zu sein
- Akustische Immersion
 - beispielsweise durch räumlichen Klang, welcher die Größe und Materialien des Raumes widerspiegelt (Hall, Dämpfung, Richtungsinformationen, Hintergrundgeräusche)
- Interaktive Objekte
 - welche vom Spieler bewegt werden können, oder Informationen bei Berührung freischalten, damit die Umgebung lebendig wirkt
- Natürlich wirkendes Licht
 - das dem realen Raum nachempfunden ist um die Orientierung zu erleichtern
- Lebensechte Skalierung
 - ermöglicht die vollständige Immersion in den Raum

8.2 Bewegungssystem

Die Fortbewegung ist ein zentraler Bestandteil der Spielerfahrung und beeinflusst das Komfortlevel als auch die Orientierung:

- Smooth Locomotion
 - schafft für erfahrene Nutzer ein natürliches Bewegungserlebnis
- Natürliche Körperbewegung
 - fördert ein natürliches Explorationsgefühl

8.3 UI/UX Design

Die Gestaltung der Benutzeroberfläche in VR unterscheidet sich deutlich von klassischen 2D-Anwendungen. Es wird besonders auf eine intuitive und minimalistische Gestaltung gesetzt, welche die Immersion des Nutzers nicht behindert.

- In-World UI
 - Informationen werden bevorzugt in die Umgebung integriert (Schilder, Aushänge, NPCs statt schwebenden TextBoxen)
- Minimale, klare Symbole
 - die schnell in ihrem Inhalt erfassbar sind und optisch mit der Umgebung harmonisieren
- Kontextsensitives UI
 - welches nur erscheint, wenn es nötig ist
- Einführung durch Interaktion
 - Nutzer lernen Steuerung schrittweise kennen, indem sie einfache Aufgaben in der Umgebung umsetzen
- Erreichbarkeit und Barrierefreiheit
 - UI-Elemente sind in komfortabler Höhe platziert, gut lesbar und nicht überladen

Das Ziel ist eine „natürliche“ Bedienung welche gleichzeitig eine klare Orientierung in der Schnitzeljagd bietet.

8.4 Komfortmaßnahmen gegen Motion-Sickness

Da Motion Sickness für viele VR-Nutzer ein kritischer Faktor ist, werden verschiedene Maßnahmen implementiert, um Unwohlsein zu vermeiden:

- Stabile Referenzrahmen
 - feste UI-Elemente, welche dem Gleichgewichtssinn zusätzliche Orientierung ermöglichen
- Langsame Grundbewegungsgeschwindigkeit
 - optionale Anpassungsmöglichkeit ist
- Raumskalierung und Augenhöhe korrekt setzen
 - um Gefühl falscher Körperposition im Raum zu vermeiden
- Nutzerindividuelle Einstellungen
 - da Empfindlichkeiten variieren

9. SPIELVERLAUF

Der Spielverlauf ist wie eine Schnitzeljagd aufgebaut. Man sucht und erledigt eine Mission nach der anderen, um den Campus besser kennenzulernen.

9.1 Grundstruktur: Lineare Missionskette

Das Spiel besteht aus **drei Hauptphasen**, welche man nacheinander freischaltet. Dabei erlangst du ein allgemeines Grundwissen und Orientierung, bis hin zur spezifischen Erkundung eines Fachbereiches.

Phase	Ort	Fokus & Lernziel	Kernelemente & Interaktion	Dauer	Priorität
1	VR/MoCap Labor	Erkundung der Technologien, zur Gewinnung von Einblicken in mögliche zukünftige Studieninhalte	Erkundung mit spezifischen möglichen hybriden 3D-Interaktionen	10min	Zuerst
2	Bibliothek	Kennenlernen der Räumlichkeiten sowie Angebote in der Bibliothek	- Interaktive Elemente (Buch) - Informationsübermittlung per UI-Elemente	10min	Definitiv
3	Mensa Academic	Kennenlernen der Mensa (inkl. Ablauf des Bezahlvorgangs)	„Guthabendilemma“ 3D Spatial Audio (Simulation der Geräuschkulisse)	10min	Definitiv

9.2 Core-Game-Loop

Der folgende Core-Game-Loop beschreibt den grundlegenden Zyklus der Spieleraktivitäten, welcher sich kontinuierlich wiederholt und das spielerische Vorankommen strukturiert.

Phase	Aktion/Verhalten des Spielers	Belohnung/Ergebnis
1. Entschlüsseln	Der Spieler Löst ein Rätsel/Hinweis um die Quest zu interpretieren	Verstehen der Quest, Richtung zum nächsten Ort
2. Suchen & Bewegen	Der Spieler bewegt sich im Spiel zu den Ergebnissen die er vorher erlangt hat	Auffinden des gesuchten Objekts oder des nächsten Hinweises.
3. Bestätigen/Sammeln	Der Spieler führt am gefundenen Ort die Quest aus	Freischalten für das nächste Level (Beginn Phase 1)

9.3 Fortbewegung und Übergänge

Es ist vorgesehen keine offene Welt zu schaffen, die Spieler werden gezielt nur durch die „wichtigsten“ Bereiche geführt.

- Ingame-Bewegung
 - Es ist eine freie **First-Person-Perspektive** in den jeweiligen 3D GS-Räumen vorgesehen
- Phasenwechsel
 - Zwischen den 3 Hauptphasen wird man mithilfe eines Teleporters weitergeleitet
- Freischaltung
 - Die Weiterleitung zum nächsten Raum wird erst eröffnet, wenn man die jeweiligen Aufgaben im Aktuellen Raum erfolgreich gemeistert hat

9.4 Abschluss

Wenn man die Quest erfolgreich löst erhält man das "**Campus Mental Map**"-Abzeichen. Dadurch wird dem Spieler übermittelt, dass er die vermittelten Orte des Campus perfekt kennt.

10. STORY & SETTING

Das gesamte Spiel beschäftigt sich mit der **Hochschule für Technik, Wirtschaft und Kultur (HTWK) Leipzig**, besser gesagt in strategisch ausgewählten, hochrelevanten Räumen der Fakultät Informatik/Medien.

10.1 Kern-Setting

Ein fotorealistischer Nachbau der realen Umgebung. Die Räume werden mithilfe der **3D Gaussian Splatting (3D GS)**-Technologie abgebildet, wodurch eine sehr Realitätsnahe Umgebung geschaffen wird, welche den Übergang in die Realität vereinfacht.

10.2 Wichtige Orte (Levels)

- VR/MoCap Labor
 - Der Startpunkt, der als technologischer und fachlicher Showcase dient
- Bibliothek
 - Kennenlernen der Räumlichkeiten sowie Angebote in der Bibliothek
- Mensa Academica
 - Dient dazu das soziale Leben auf dem HTWK-Campus zu veranschaulichen und den technischen Ablauf beim aufladen von Mensaguthaben zu zeigen

10.3 Story & Motivation

Das Ziel ist nicht eine Komplexe Erzählung, eher auf einfachen klaren Lektionen, welche die Hauptzielgruppen anspricht.

Prämisse:

Der Spieler ist ein neuer Student oder Interessent an der HTWK, welcher einfach ohne äußeren Einfluss den Campus kennenlernen möchte.

Die Quest:

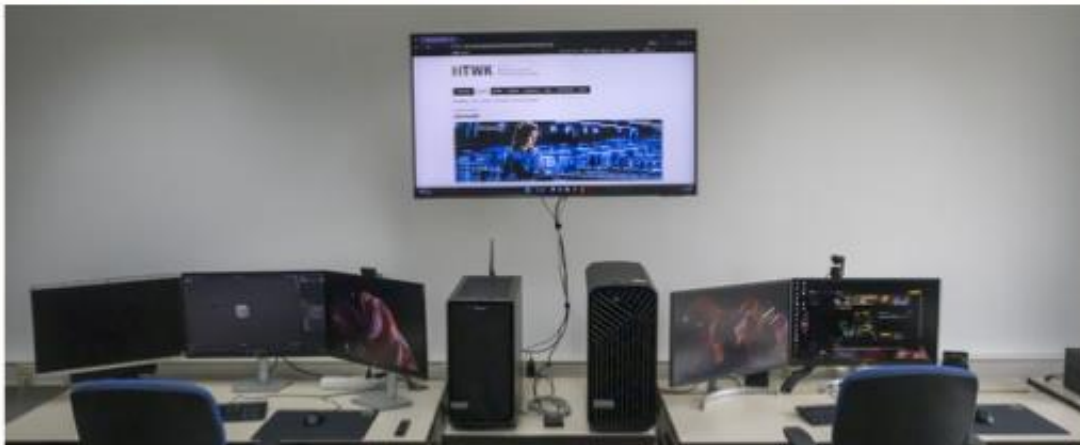
Das Herzstück der Story ist die „Campus-Einweihungs-Quest“, eine Art ineraktive Schnitzeljagd. Hierbei müssen die Spieler durch ausprobieren und erkunden den Campus näher kennenlernen und Wissen über ihn erlangen.

Ziel des Spielers:

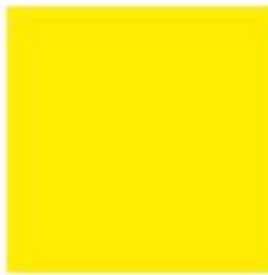
Wenn man die Quest erfolgreich löst, erhält man das "**Campus Mental Map**"-Abzeichen. Dadurch wird dem Spieler übermittelt, dass er die vermittelten Orte des Campus perfekt kennt.

11. MOODBOARDS UND IMPRESSIONEN

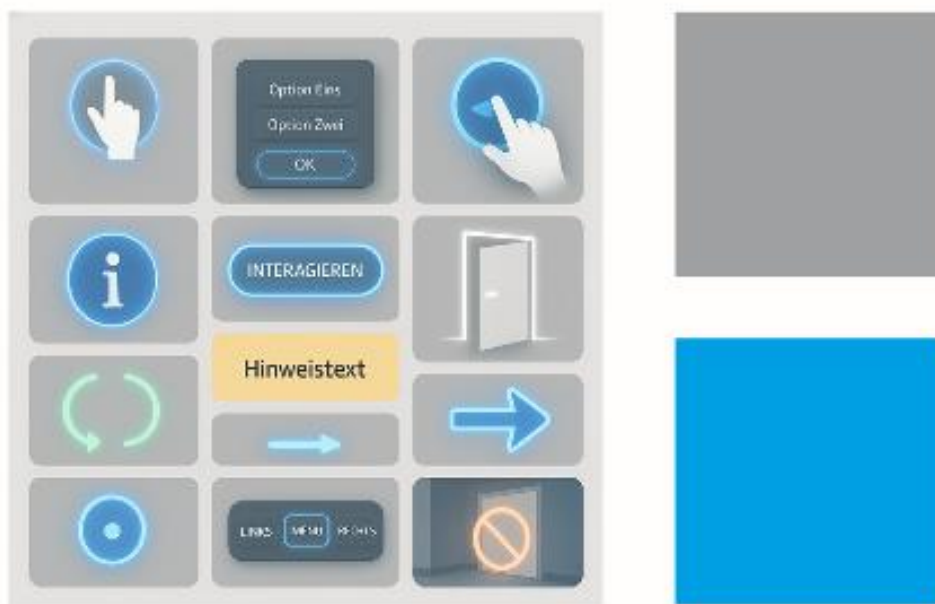
11.1 Räume der HTWK Leipzig



11.2 Farb- und Lichtabstimmung



11.3 UI / UX und interaktive Elemente



13 QUELLEN

13.1 Bildquellen

- https://www.e-learning.tu-darmstadt.de/media/elc/___relaunch/03_werkzeuge/360_ar_vr_lernumgebungen/Buehne_360Grad_AR_VR_Lernumgebungen_1300x0.jpg
- https://vrlab-htwk.de/wp-content/uploads/2023/02/IMG_0926-1-1200x700.jpeg
- https://vrlab-htwk.de/wp-content/uploads/2023/02/Raum_komplett-2048x1150.jpg
- https://www.studentenwerk-leipzig.de/ext/uploads/2024/01/IMG_3454-scaled.jpg
- <https://www.studentenwerk-leipzig.de/ext/uploads/2024/01/4096-2621-max-scaled.jpg>
- https://www.htwk-leipzig.de/fileadmin/_processed_/9/b/csm_Mensa_839c75f549.jpg
- https://www.htwk-leipzig.de/fileadmin/_processed_/0/4/csm_Hochschulbibliothek_Reichhold20160621_32_76f94a1bca.jpg
- <https://freesound.org/people/NazarHK/sounds/787575/>
- <https://freesound.org/people/Pfannkuchn/sounds/457604/>
- Nutzung von KI generierten Bildern